EEN025 Assignment 3

Group 6

October 19, 2026

Task 1: The Tower of Hanoi

Model

We model the puzzle with n disks (disk 0 is the smallest), m towers and k time-steps (moves) as a pure SAT problem in Z3. The decision question is: *can the stack be moved from tower* 0 *to tower* $m-1$ *in exactly* k *moves?*

All variables are Boolean.

Variable	Range	Meaning
(d, w, t)	$d < n,\ w < m,\ t = 0..k$	disk d is on tower w at time t
(d, t)	$d < n,\ t = 0..k-1$	disk d is moved at step t
(w, t)	$w < m,\ t = 0..k-1$	the moved disk is put on tower w at step t

Constraints

For all $t \in [0, k-1]$, $d \in [0, n-1]$, $w \in [0, m-1]$:

1. **Initial and final state.** $(d, 0, 0)$ and $(d, m-1, k)$ hold for every disk, and $(d, w, 0)$, (d, w, k) are false on all other towers.
2. **One move per step.** Exactly one disk is moved and exactly one destination is chosen: AtMost1 and AtLeast1 over $(\cdot, t)$ and over $(\cdot, t)$.
3. **Precondition I (moved disk is on top).** $(d, t) \wedge (d, w, t) \rightarrow \neg \bigvee_{e<d}(e, w, t)$.
4. **Precondition II (no smaller disk at the destination).** $(d, t) \wedge (w, t) \rightarrow \neg \bigvee_{e<d}(e, w, t)$.
5. **Source $\neq$ destination.** $(d, t) \wedge (d, w, t) \rightarrow \neg(w, t)$.
6. **Update and frame axiom in one equivalence.**

$$(d, w, t{+}1) \ \leftrightarrow\ \mathrm{ite}\big((d, t),\ (w, t),\ (d, w, t)\big).$$

Constraint 6 replaces the attachment's constraints (8) and (10); it also keeps every disk on exactly one tower, so no separate uniqueness constraint is needed.

Results

We used $m = 3$ towers, $k = 2^n - 1$, and also $k = 2^n - 2$ to show that one step less is infeasible. Each plan returned by Z3 was replayed by an independent checker that verifies that every move is legal and the final state is reached.

n	$k = 2^n - 1$	result	time [s]	$k - 1$	result	time [s]
3	7	sat	0.12	6	unsat	0.03
4	15	sat	0.11	14	unsat	0.09
5	31	sat	0.61	30	unsat	0.33
6	63	sat	1.44		not run	
7	127	sat	47.39		not run	
8	255	run or state not finished				

Table 1: Feasibility for $m = 3$ towers.

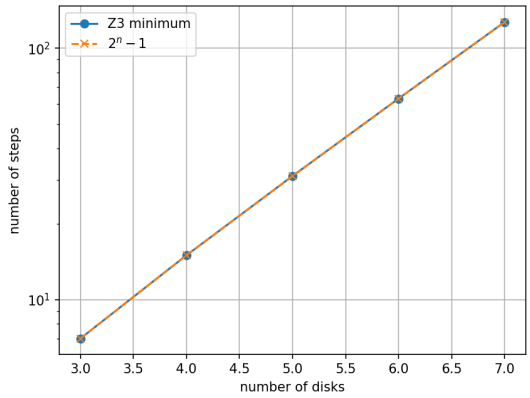


Figure 1: Minimal number of steps versus number of disks (log scale), with $2^n - 1$.

The minimum found by Z3 equals $2^n - 1$ for every n, matching Equation (1) of the assignment. The number of steps roughly doubles per disk, and the solving time grows much faster: from $n = 5$ to $n = 7$ the steps grow by a factor 4 but the time by a factor of about 78 (0.61 s $\rightarrow$ 47.39 s). The proofs that $k - 1$ steps are not enough were quick for $n \leq 5$.

Finding the minimum without knowing $2^n - 1$

Idea. Fix a horizon K that is only an upper bound and allow *idle* steps in which nothing moves. Constraint 2 becomes $\bigvee_d (d, t) \leftrightarrow \bigvee_w (w, t)$ (with at most one of each), and idle steps are only allowed at the end. The objective is to minimize the number of real moves, $\sum_t \left[\bigvee_d (d, t) \right]$. A bound that does not use the formula is $K = m^n - 1$, since a shortest plan never repeats a state and there are m^n states (the experiments used $K = 2^n + 3$ to keep runs short).

We compared three methods: **iterative deepening** (solve for $k = 1, 2, \ldots$ until the first `sat`), **bisection** on K (valid because with idle steps "solvable within K steps" is monotone in K), and Z3's `Optimize` with the objective above.

n	Iterative deepening [s]	Bisection [s]	`Optimize` [s]	minimum
3	0.12	0.12	0.06	7
4	0.73	0.46	0.13	15
5	4.91	1.58	0.48	31

Table 2: Methods for finding the minimal number of steps.

All three return 7, 15 and 31, i.e. $2^n - 1$, without the formula being given to the solver. Iterative deepening is slowest, since it must prove `unsat` for every k below the optimum; bisection needs about $\log_2 K$ solver calls; `Optimize` was fastest, about $10\times$ faster than iterative deepening for $n = 5$.

Conclusion

The parametrized Boolean model finds a plan with $2^n - 1$ steps for $n = 3, \ldots, 7$ disks on three towers and proves for $n = 3, 4, 5$ that one step less is infeasible, so the minimal number of steps is $2^n - 1$ in these cases. Introducing idle steps and an objective lets Z3 find this minimum without being given the formula; `Optimize` was the fastest of the three methods tested. The solving time grows much faster than the plan length, which limits the size of instances that can be solved with this formulation.

Task 2: A General Sorting Problem

Model Definition

Let B be the set of bricks, P the set of positions, and let

$$t = 0, 1, \ldots, H$$

be the time horizon. Each brick b has an initial position $\mathit{initial}(b) \in P$ and a goal position $\mathit{goal}(b) \in P$.

The main state variable is

$$\mathit{at}(b, p, t),$$

which is true when brick b is at position p at time t.

The action variable is

$$\mathit{move}(b, p, q, t),$$

which is true when brick b moves from position p to position q during time-step t, where $p \neq q$.

The variable

$$\mathit{moved}(b, t)$$

indicates whether brick b is moved at time t.

Constraints

Unique Position

Every brick must occupy exactly one position:

$$\sum_{p \in P} at(b, p, t) = 1 \qquad \forall b \in B,\ \forall t.$$

Every position can contain at most one brick:

$$\sum_{b \in B} at(b, p, t) \leq 1 \qquad \forall p \in P,\ \forall t.$$

The second constraint is the general replacement for the Tower of Hanoi disk ordering rule.

Initial and Goal States

If $p_0 = initial(b)$, then

$$at(b, p_0, 0) = True.$$

If $p_g = goal(b)$, then

$$at(b, p_g, H) = True.$$

Since each brick has exactly one position, these constraints also exclude all other positions in the corresponding states.

Movement and Transition

At most one brick can be moved per time-step:

$$\sum_{b \in B} moved(b, t) \leq 1 \qquad \forall t = 0, \ldots, H - 1.$$

A brick can only move from its current position:

$$move(b, p, q, t) \Rightarrow at(b, p, t).$$

If a brick is not moved, it remains at the same position:

$$at(b, p, t) \wedge \neg moved(b, t) \Rightarrow at(b, p, t + 1).$$

If it moves from p to q, it arrives at q at the next time-step:

$$move(b, p, q, t) \Rightarrow at(b, q, t + 1).$$

Together with the unique-position constraints, these rules define the complete state transition.

Objective Function

The total number of movements is

$$TotalMoves = \sum_{b \in B} \sum_{t=0}^{H-1} moved(b, t).$$

The objective is

$$\min TotalMoves.$$

The program searches increasing values of H. The first feasible horizon is the minimum number of movements because only one brick can move in each time-step.

Example

Consider three bricks and three positions:

$$initial(b_1) = p_1, \quad initial(b_2) = p_2, \quad initial(b_3) = p_3,$$

and

$$goal(b_1) = p_3, \quad goal(b_2) = p_1, \quad goal(b_3) = p_2.$$

One optimal plan is

$$b_1 : p_1 \to p_3,$$

$$b_2 : p_2 \to p_1,$$

$$b_3 : p_3 \to p_2.$$

The final state is

$$b_1 \to p_3, \qquad b_2 \to p_1, \qquad b_3 \to p_2,$$

and the minimum number of movements is

$$TotalMoves = 3.$$

Scalability

Let B be the number of bricks, P the number of positions, and H the planning horizon. The state variables require approximately

$$O(BPH)$$

variables. The direct movement representation requires approximately

$$O(BP^2H)$$

variables because every source-destination pair is considered.

The number of possible configurations is

$$\frac{P!}{(P-B)!}.$$

Thus, the search space grows rapidly with the number of bricks and positions. This is the main source of state-space explosion.

Possible improvements include:

1. using integer position variables;
2. using `Distinct` instead of many pairwise constraints;
3. removing symmetric solutions;
4. reducing the planning horizon;
5. using a heuristic solution as an upper bound.

Identical Bricks

If bricks of the same colour are interchangeable, individual identities do not need to be represented. A counting variable can be used:

$$count(c, p, t),$$

where c is a colour. If N_c is the total number of bricks of colour c, then

$$\sum_{p \in P} count(c, p, t) = N_c.$$

This removes symmetric solutions and may improve scalability. However, it cannot be used when each brick has an individual target position.

Multiple Robots and Platforms

For multiple robots, the action variable can be extended to

$$move(r, b, p, q, t),$$

where r identifies the robot. A robot-position variable $robotAt(r, p, t)$ may also be added.

The extended model must ensure that robots do not move two bricks at once, that one brick is not moved by two robots, that platforms have sufficient capacity, and that robots can reach the required positions. This increases the number of variables and possible conflicts.

Distance Objectives

If positions have coordinates $p = (x_p, y_p)$, the Manhattan distance is

$$d_M(p, q) = |x_p - x_q| + |y_p - y_q|.$$

The Euclidean distance is

$$d_E(p, q) = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2}.$$

For example, the total Manhattan distance is

$$D_M = \sum_{b,t,p,q} d_M(p, q)\, move(b, p, q, t).$$

A combined objective can be defined as

$$\min\left(\alpha TotalMoves + \beta D\right),$$

where α and β determine the importance of movements and distance.

Conclusion

Task II generalizes the planning model from Task I to a robot sorting problem. The state variables describe brick positions, and the action variables describe movements. The constraints enforce unique positions, collision avoidance, legal transitions, and the final goal configuration.

The direct model grows as $O(BP^2H)$, while the number of possible configurations grows combinatorially. Therefore, larger instances may suffer from state-space explosion. Integer variables, symmetry breaking, `Distinct`, and colour-based modelling can improve the model. The same framework can also represent multiple robots and distance-based objectives.